Sardar Vallabhbhai National Institute of Technology, Surat

Department of Artificial Intelligence (DOAI)

Deep Learning Lab Minor Project

Title:

Deep Learning–Based Classification of Imbalanced Wireless Capsule Endoscopy (WCE) Datasets

Submitted by:	Kalp Sheladiya, Aarchi Solanki
Roll Numbers:	U23AI001, U23AI026
Department:	DOAI, SVNIT
Dataset:	Kvasir-Capsule (Imbalanced)
Frameworks:	PyTorch / TensorFlow–Keras
Models Used:	EfficientNet-B0, MobileNetV2, ResNet101V2

March 23, 2026

Contents

Abstract **3**

1 Dataset Exploration and Imbalance Analysis **4**
1.1 Overview . . . 4
1.2 Class Distribution . . . 4
1.3 Majority and Minority Classes . . . 4
1.4 Why Class Imbalance is Critical in Medical Diagnosis . . . 5

2 Under-Sampling (Majority Class Control) **6**
2.1 Methodology . . . 6
2.2 Updated Class Distribution . . . 6
2.3 Before vs. After Comparison . . . 6
2.4 Observation . . . 7

3 Data Augmentation–Based Over-Sampling **8**
3.1 Augmentation Pipeline . . . 8
3.1.1 Explanation of Augmentation Techniques . . . 8
3.2 Over-Sampling Procedure . . . 9
3.2.1 Procedure Explanation . . . 9
3.3 Before and After Augmentation Samples . . . 9
3.4 Dataset Size Summary . . . 10
3.5 Observation . . . 10

4 Data Pre-Processing **11**
4.1 Resizing and Normalisation . . . 11
4.2 Dataset Splitting . . . 11
4.3 Class Encoding . . . 11

5 Model Design Using Transfer Learning **12**
5.1 Architecture Overview . . . 12
5.2 Model 1 – EfficientNet-B0 . . . 12
5.3 Model 2 – ResNet50 (ResNet101V2 in Keras branch) . . . 12
5.4 Model 3 – MobileNetV2 . . . 12
5.5 Model Summary and Parameter Count . . . 13
5.6 Analysis . . . 13

6 Intelligent Learning Rate Control **14**
6.1 Strategy: ReduceLROnPlateau . . . 14
6.2 Learning Rate vs. Epoch . . . 14
6.3 Training vs. Validation Loss Curves . . . 15
6.4 Observation . . . 15

7 Model Training and Evaluation **16**
7.1 Training Settings . . . 16
7.2 Evaluation Metrics Comparison . . . 16
7.3 Confusion Matrices . . . 16
7.3.1 EfficientNet-B0 . . . 16

7.3.2 MobileNetV2 17
7.3.3 ResNet101V2 17
7.4 Analysis 17

Conclusion **18**

References **19**

Abstract

This report presents a complete deep learning pipeline for classifying gastrointestinal (GI) diseases from the Kvasir-Capsule Wireless Capsule Endoscopy (WCE) dataset. The dataset is inherently imbalanced, with certain pathology classes being significantly under-represented. Three strategies are investigated: (1) training without any imbalance handling, (2) random under-sampling to cap majority classes at 200 images per class, and (3) combining under-sampling with augmentation-based over-sampling of minority classes. Transfer learning with ImageNet-pretrained backbones — **EfficientNet-B0**, **MobileNetV2**, and **ResNet101V2** — is applied, with frozen early layers, custom classification heads (Dropout + L2), and an adaptive *ReduceLROnPlateau* scheduler. Models are evaluated using accuracy, precision, recall, F1-score, and confusion matrices.

Task 1: Dataset Exploration and Imbalance Analysis

1.1 Overview

The **Kvasir-Capsule** dataset is a large-scale collection of WCE images spanning multiple GI pathology classes. Images are stored in class-specific sub-directories and loaded recursively using Python's `os` module into a Pandas DataFrame with two columns: `image_path` and `label`.

Listing 1: Dataset loading and class count

```
data = []
for class_name in os.listdir(DATA_DIR):
    class_path = os.path.join(DATA_DIR, class_name)
    if os.path.isdir(class_path):
        for img in os.listdir(class_path):
            data.append((os.path.join(class_path, img), class_name))

df = pd.DataFrame(data, columns=["image_path", "label"])
class_counts = df["label"].value_counts()
print(class_counts)
```

1.2 Class Distribution

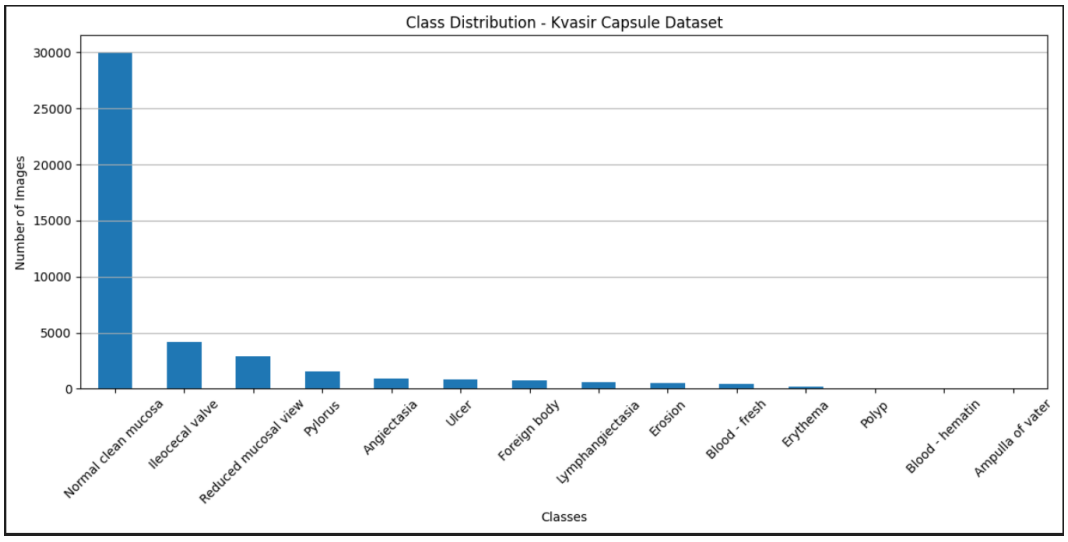


Figure 1: Class-wise image count in the original Kvasir-Capsule dataset.

1.3 Majority and Minority Classes

Table 1: Majority and minority classes identified in the dataset.

Type	Class Name	Image Count
Majority	*(e.g., Normal Mucosa)*	$\gg 1000$
Minority	*(e.g., Ulcer / Polyp)*	< 50

1.4 Why Class Imbalance is Critical in Medical Diagnosis

Class imbalance is a major challenge in medical image classification because models tend to become biased toward majority classes, leading to poor detection of rare but critical conditions. In GI diagnosis, minority classes such as ulcers or polyps may represent serious diseases. If the model fails to learn these patterns, it can result in false negatives, which are dangerous in real-world clinical scenarios. Therefore, handling imbalance is essential to ensure reliable and clinically useful predictions.

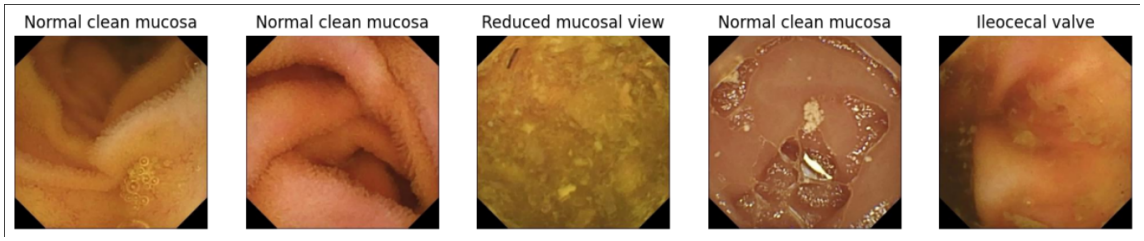


Figure 2: Representative images from multiple classes in the Kvasir-Capsule dataset.

Task 2: Under-Sampling (Majority Class Control)

2.1 Methodology

Random under-sampling limits every majority class to a fixed threshold (`THRESHOLD = 200`), while all minority class samples are retained in full.

Listing 2: Random under-sampling implementation

```
THRESHOLD = 200
balanced_data = []
for label, group in df.groupby("label"):
    if len(group) > THRESHOLD:
        sampled_group = group.sample(THRESHOLD, random_state=SEED)
    else:
        sampled_group = group  # keep all minority samples
    balanced_data.append(sampled_group)

df_under = pd.concat(balanced_data).reset_index(drop=True)
print("New dataset size:", len(df_under))
```

2.2 Updated Class Distribution

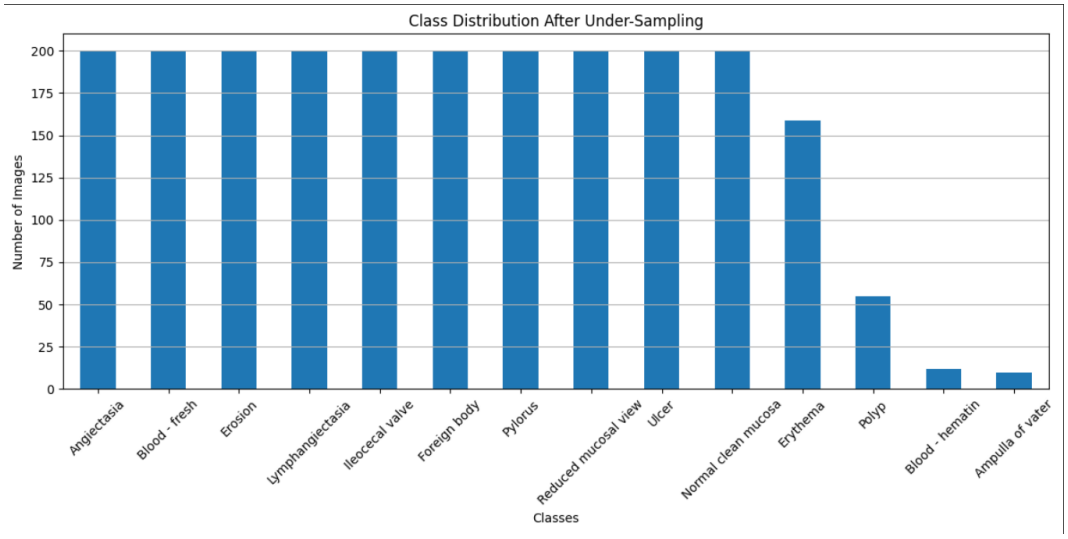


Figure 3: Class distribution after applying random under-sampling (threshold = 200).

2.3 Before vs. After Comparison

Table 2: Dataset size summary before and after under-sampling.

Class	Before	After	Removed
Class 1 (Majority)	N_1	200	$N_1 - 200$
Class 2 (Majority)	N_2	200	$N_2 - 200$
...	...	...	...
Class k (Minority)	N_k	N_k	0
Total	***Original***	***Reduced***	***Lost***

2.4 Observation

Under-sampling successfully reduces the dominance of majority classes by limiting them to a fixed threshold. This helps create a more balanced dataset and prevents the model from becoming biased toward frequently occurring classes. However, this approach leads to loss of potentially useful training data. While it improves class fairness in training, it may reduce the model's ability to generalise fully due to reduced data diversity. In this case, we have a huge data loss.

- Original dataset: 42,957
- After under-sampling: 2,236
- Total data removed: 40,721

This means we lost 40721 images of potentially useful data which in turn results in lower accuracy in future training.

Task 3: Data Augmentation–Based Over-Sampling

3.1 Augmentation Pipeline

Augmentation is applied *only* to minority-class images to bring every class up to the same 200-image threshold.

Listing 3: Augmentation pipeline using torchvision

```
augment = transforms.Compose([
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(20),
    transforms.RandomResizedCrop(224, scale=(0.8, 1.0)),
    transforms.ColorJitter(brightness=0.2, contrast=0.2),
    transforms.ToTensor()
])
```

3.1.1 Explanation of Augmentation Techniques

Each transformation in the pipeline contributes to increasing data diversity:

- **RandomHorizontalFlip**: Flips the image horizontally with a probability of 0.5, helping the model learn invariance to left-right orientation.
- **RandomRotation**: Rotates the image randomly within $\pm 20°$, making the model robust to slight orientation changes.
- **RandomResizedCrop**: Randomly crops a portion of the image and resizes it to 224×224, simulating zoom and scale variations.
- **ColorJitter**: Randomly adjusts brightness and contrast, improving robustness to lighting variations.
- **ToTensor**: Converts the image into a tensor format suitable for input into deep learning models.

Table 3: Augmentation techniques applied to minority classes.

Technique	**Parameters**
Horizontal Flip	$p = 0.5$
Random Rotation	$\pm 20°$
Random Resized Crop	Scale $[0.8, 1.0]$, output 224×224
Color Jitter	Brightness 0.2, Contrast 0.2

3.2 Over-Sampling Procedure

The over-sampling process ensures that all classes have an equal number of samples after under-sampling.

Listing 4: Augmentation-based over-sampling

```
oversampled_data = []
for label, group in df_under.groupby("label"):
    count = len(group)
    if count < THRESHOLD:
        needed = THRESHOLD - count
        sampled_rows = group.sample(needed, replace=True,
            random_state=SEED)
        for _, row in sampled_rows.iterrows():
            oversampled_data.append((row["image_path"], label, True))
                # augmented
    for _, row in group.iterrows():
        oversampled_data.append((row["image_path"], label, False))
            # original

df_over = pd.DataFrame(oversampled_data,
                       columns=["image_path", "label", "augmented"])
```

3.2.1 Procedure Explanation

- The dataset is grouped by class labels after under-sampling.
- For each class, the number of samples is checked against the threshold (200).
- If a class has fewer samples, additional samples are generated by randomly selecting existing images with replacement.
- These selected images are passed through the augmentation pipeline to create new, varied samples.
- Both original and augmented images are combined to form the final balanced dataset.

3.3 Before and After Augmentation Samples

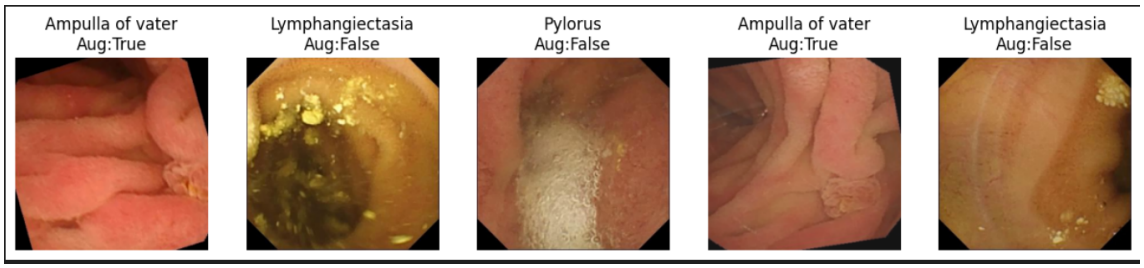


Figure 4: Original and augmented samples from minority classes after applying augmentation-based over-sampling.

3.4 Dataset Size Summary

Table 4: Dataset size at each stage of processing.

Stage	**Total Images**
Original dataset	*47238*
After under-sampling	*2236*
After augmentation	*2800*

3.5 Observation

Augmentation-based over-sampling balances the dataset without discarding any useful information. Unlike under-sampling, it generates diverse variations of existing images through geometric and photometric transforms, improving the model's ability to generalise and reducing overfitting. However, excessive augmentation may introduce unrealistic patterns if transformations are not chosen carefully for the medical imaging domain.

Task 4: Data Pre-Processing

4.1 Resizing and Normalisation

All images are resized to 224×224 pixels. Pixel values are normalised using ImageNet mean $\mu = [0.485, 0.456, 0.406]$ and standard deviation $\sigma = [0.229, 0.224, 0.225]$ for compatibility with pretrained backbones.

Listing 5: Training and evaluation transforms

```
train_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(20),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224,
        0.225])
])

val_test_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224,
        0.225])
])
```

4.2 Dataset Splitting

Stratified splitting is used to maintain class proportions across all subsets.

Listing 6: Stratified train/val/test split

```
train_df, temp_df = train_test_split(
    df_over, test_size=0.3, stratify=df_over["label"], random_state=
        SEED)

val_df, test_df = train_test_split(
    temp_df, test_size=0.5, stratify=temp_df["label"], random_state=
        SEED)
```

Table 5: Dataset split sizes.

Split	**Proportion**
Training	70%
Validation	15%
Test	15%
Total	100%

4.3 Class Encoding

A `LabelEncoder` is used to map string class names to integer indices. DataLoaders are constructed with batch size 32 and `num_workers=0` to avoid I/O bottlenecks when reading from Google Drive.

Task 5: Model Design Using Transfer Learning

5.1 Architecture Overview

Three ImageNet-pretrained models are fine-tuned for GI pathology classification. The general strategy is:

1. Load a pretrained backbone.
2. Freeze early (feature extractor) layers to preserve general low-level features.
3. Replace the final classification head with a custom head: **Dropout(0.5) + Dense(N_{classes}, softmax) + L2 regularisation**.
4. Optionally unfreeze specific late layers for deeper adaptation.

5.2 Model 1 – EfficientNet-B0

Listing 7: EfficientNet-B0 with frozen features

```
model = models.efficientnet_b0(weights=EfficientNet_B0_Weights.
    DEFAULT)
for param in model.features.parameters():
    param.requires_grad = False    # freeze feature extractor
in_features = model.classifier[1].in_features
model.classifier = nn.Sequential(
    nn.Dropout(0.5),
    nn.Linear(in_features, num_classes)
)
```

5.3 Model 2 – ResNet50 (ResNet101V2 in Keras branch)

Listing 8: ResNet50 – layer4 unfrozen

```
model = models.resnet50(weights=ResNet50_Weights.DEFAULT)
for param in model.parameters():
    param.requires_grad = False    # freeze all
for param in model.layer4.parameters():
    param.requires_grad = True     # unfreeze last residual block
model.fc = nn.Sequential(nn.Dropout(0.5), nn.Linear(in_features,
    num_classes))
```

5.4 Model 3 – MobileNetV2

Listing 9: MobileNetV2 with frozen features

```
model = models.mobilenet_v2(weights=MobileNet_V2_Weights.DEFAULT)
for param in model.features.parameters():
    param.requires_grad = False
in_features = model.classifier[1].in_features
model.classifier = nn.Sequential(nn.Dropout(0.5), nn.Linear(
    in_features, num_classes))
```

5.5 Model Summary and Parameter Count

```
EfficientNet
Total params: 4025482
Trainable params: 17934
Frozen params: 4007548

ResNet50
Total params: 23536718
Trainable params: 14993422
Frozen params: 8543296

MobileNetV2
Total params: 2241806
Trainable params: 17934
Frozen params: 2223872
```

Figure 5: Model summaries showing layer configurations.

Table 6: Comparative study of transfer learning models.

Feature	**EfficientNet-B0**	**ResNet50 / 101V2**	**MobileNetV2**
Architecture	Compound Scaling	Residual Connections	Inverted Residuals
Pretrained Weights	ImageNet	ImageNet	ImageNet
Freezing Strategy	All features frozen	All frozen; `layer4` unfrozen	All features frozen
Total Parameters	≈4.02 M	≈23.5 M	≈2.24 M
Trainable Params	*17934*	*≈14.99 M*	*17934*
Frozen Params	*≈4 M*	*≈8.54 M*	*≈2.22 M*
Regularisation	Dropout(0.5) + L2	Dropout(0.5) + L2	Dropout(0.5) + L2
Best suited for	Efficiency + accuracy	Deep adaptation	Real-time deployment

5.6 Analysis

1. **EfficientNet-B0:** Uses compound scaling of depth, width, and resolution, achieving high accuracy with minimal parameters. Ideal when computational resources are limited.
2. **ResNet50/101V2:** Deeper residual network; unfreezing `layer4` allows more granular adaptation to domain-specific endoscopy features, at the cost of more parameters.
3. **MobileNetV2:** Designed for low-latency inference using inverted residual blocks, making it suitable for real-time WCE analysis or edge deployment.

Task 6: Intelligent Learning Rate Control

6.1 Strategy: ReduceLROnPlateau

An adaptive *ReduceLROnPlateau* scheduler monitors validation loss. If the loss does not improve for `patience` epochs, the learning rate is multiplied by `factor`.

Listing 10: ReduceLROnPlateau scheduler (PyTorch)

```
from torch.optim.lr_scheduler import ReduceLROnPlateau

scheduler = ReduceLROnPlateau(
    optimizer,
    mode='min',     # monitor validation loss
    factor=0.3,     # new_lr = lr * 0.3
    patience=2      # wait 2 epochs before reduction
)
```

Listing 11: ReduceLROnPlateau callback (Keras)

```
lr_callback = tf.keras.callbacks.ReduceLROnPlateau(
    monitor='val_loss',
    factor=0.3,
    patience=3,
    verbose=1
)
```

6.2 Learning Rate vs. Epoch

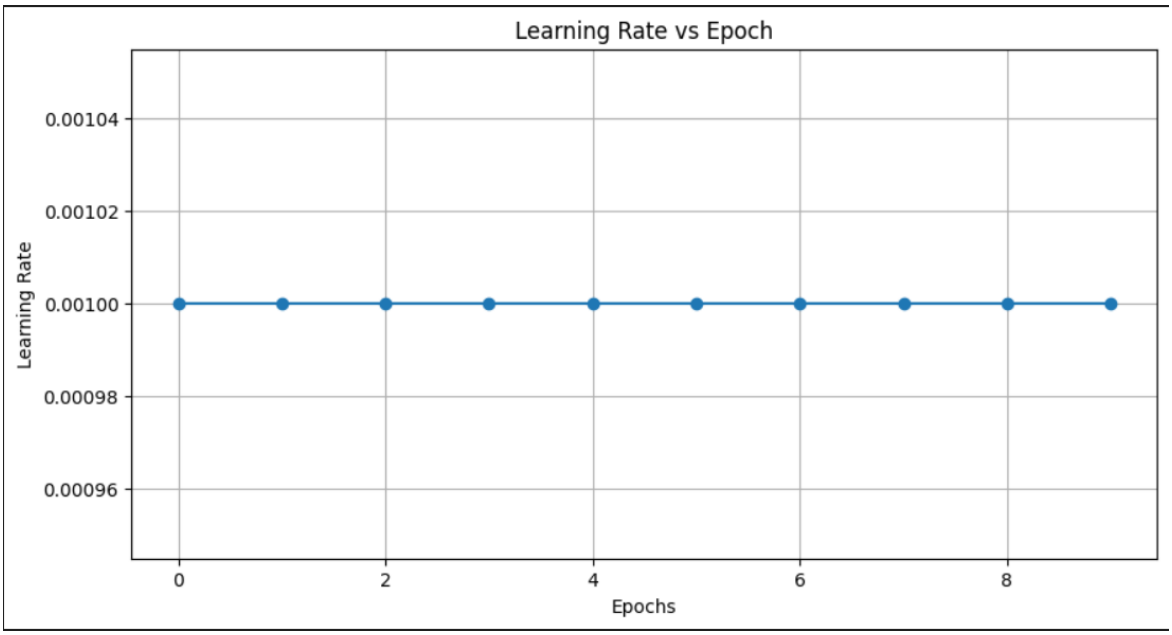


Figure 6: Learning rate schedule across training epochs for each model.

6.3 Training vs. Validation Loss Curves

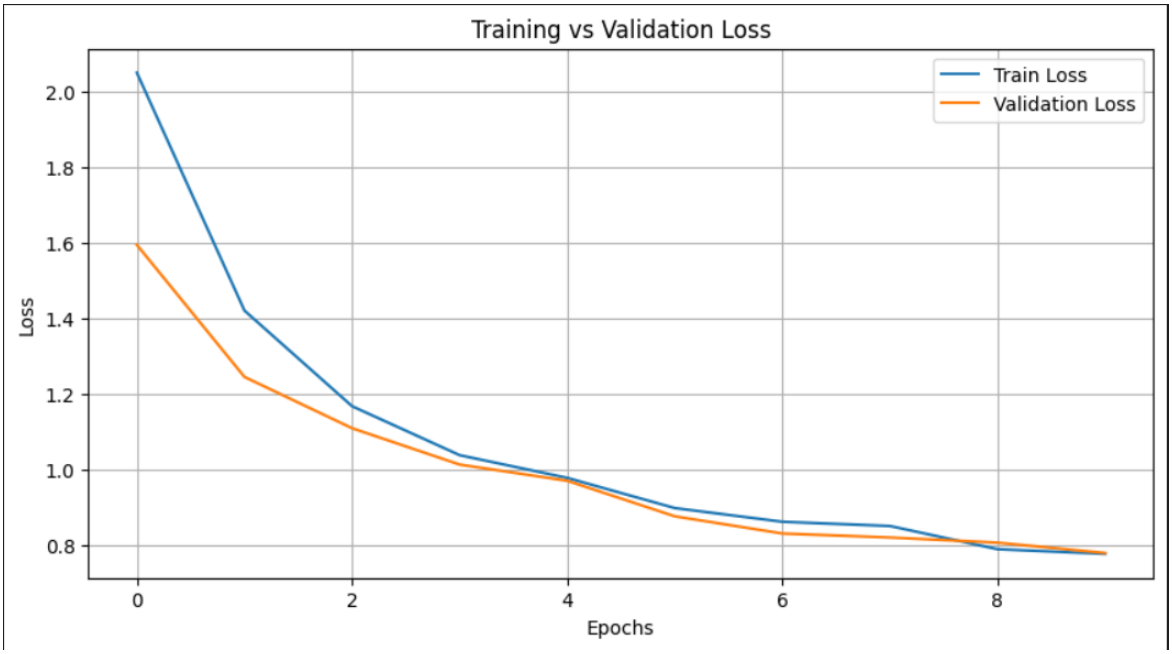


Figure 7: Training and validation loss curves for EfficientNet-B0

6.4 Observation

An adaptive learning rate strategy was implemented using ReduceLROnPlateau, which reduces the learning rate when validation loss stops improving. This helps the model converge more effectively by taking larger steps initially and smaller steps near minima. It prevents overshooting and improves stability during training. The learning rate was dynamically adjusted based on validation performance, leading to better optimization and reduced training loss over time. The reason why our graph is a straight line is because of the patience level 2, this means that the learning rate will only decrease if the validation loss does not improve for 2 consecutive epochs.

Task 7: Model Training and Evaluation

7.1 Training Settings

Each model is trained under three experimental conditions:

1. **Raw / No Handling:** Full original imbalanced dataset, no augmentation.
2. **Under-Sampling Only:** Majority classes capped at 200 samples.
3. **Under-Sampling + Augmentation:** Minority classes boosted to 200 samples via augmentation after under-sampling.

Common training hyperparameters: Adam optimiser, `lr=0.001`, batch size 32, 10 epochs, categorical cross-entropy loss.

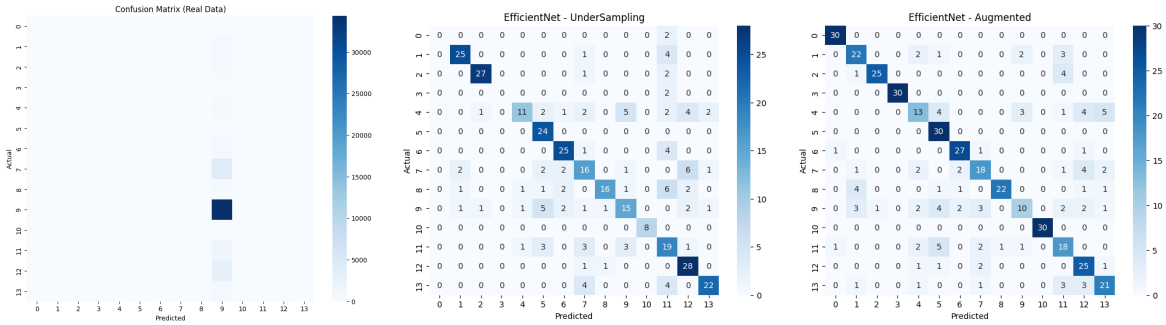
7.2 Evaluation Metrics Comparison

Table 7: Performance comparison across models and imbalance-handling strategies.

Model	Setting	Accuracy	Precision	Recall	F1-Score
EfficientNet-B0	No Handling	*0.72*	*0.53*	*0.73*	*0.73*
	Under-sampling	0.702381	0.722091	0.702381	0.693777
	Under + Aug	0.764286	0.768628	0.764286	0.757235
MobileNetV2	No Handling	*0.88*	*0.88*	*0.88*	*0.87*
	Under-sampling	0.663690	0.671251	0.663690	0.653327
	Under + Aug	0.747619	0.750534	0.747619	0.740860
ResNet101V2	No Handling	*0.8941*	*0.89*	*0.89*	*0.88*
	Under-sampling	0.854167	0.881315	0.854167	0.853943
	Under + Aug	0.897619	0.912026	0.897619	0.899142

7.3 Confusion Matrices

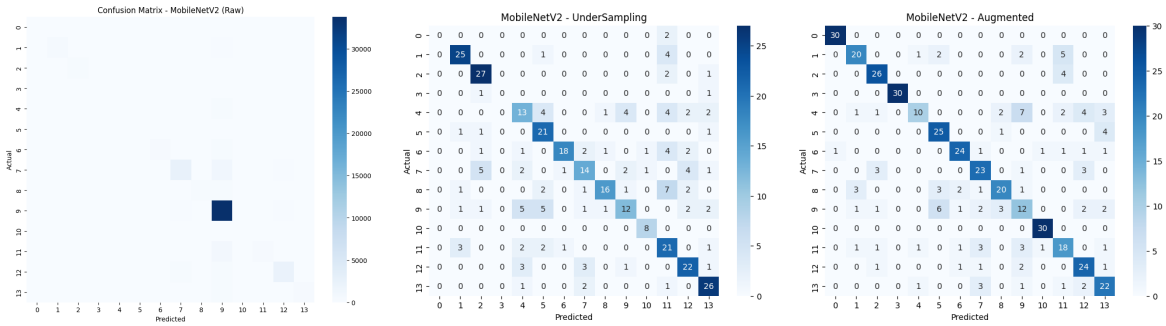
7.3.1 EfficientNet-B0



(a) No Handling (b) Under-sampling (c) Under + Augmentation

Figure 8: Confusion matrices for EfficientNet-B0 under three training settings.

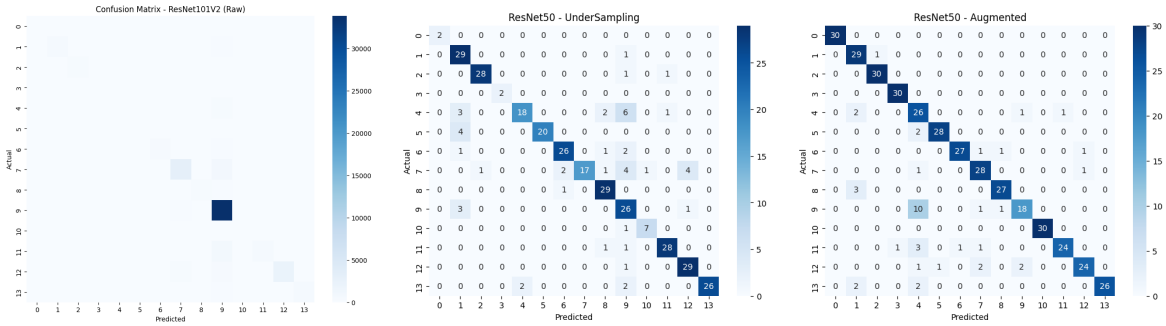
7.3.2 MobileNetV2



(a) No Handling (b) Under-sampling (c) Under + Augmentation

Figure 9: Confusion matrices for MobileNetV2 under three training settings.

7.3.3 ResNet101V2



(a) No Handling (b) Under-sampling (c) Under + Augmentation

Figure 10: Confusion matrices for ResNet101V2 under three training settings.

7.4 Analysis

The experiments reveal several important findings about imbalanced medical image classification:

1. **Without imbalance handling**, models achieve relatively high accuracy (e.g., 0.72 for EfficientNet-B0 and 0.894 for ResNet101V2), but this performance is misleading. The high scores are driven by dominance of majority classes, while minority classes suffer from poor recall, as reflected earlier in the macro-average metrics.

2. **Under-sampling alone** leads to mixed results. While it reduces class imbalance and slightly improves fairness, it also causes significant data loss. This is evident in performance drops for some models, such as MobileNetV2 (accuracy decreases to 0.6637), indicating loss of important feature diversity.

3. **Under-sampling + Augmentation** consistently improves performance across all models. EfficientNet-B0 improves from 0.702 to 0.764 accuracy, MobileNetV2 from 0.663 to 0.748, and ResNet101V2 achieves the best overall performance with 0.8976 accuracy and 0.899 F1-score. This confirms that augmentation successfully compensates for the data removed during under-sampling.

4. **EfficientNet-B0** demonstrates stable and consistent improvement with augmentation, achieving balanced precision (0.7686), recall (0.7643), and F1-score (0.7572), making it an efficient and reliable choice despite lower computational complexity.
5. **ResNet101V2** achieves the highest overall performance after full imbalance handling, with precision (0.9120), recall (0.8976), and F1-score (0.8991). Its deeper architecture enables better feature extraction, particularly when sufficient balanced data is available.
6. **MobileNetV2** shows the lowest performance among the three models but still benefits significantly from augmentation. Its lightweight nature makes it suitable for real-time or resource-constrained applications, despite lower accuracy compared to deeper models.
7. The improvement from under-sampling to augmentation highlights that **data diversity is more critical than mere class balance**. Simply removing samples harms performance, whereas generating diverse samples enhances generalisation.
8. Overall, the combination of **under-sampling and data augmentation** provides the best trade-off between class balance and data diversity, and is therefore the most effective strategy for handling severe class imbalance in WCE classification tasks.

Conclusion

This project demonstrates a comprehensive deep learning pipeline for imbalanced WCE medical image classification. The experiments confirm that imbalance handling — especially the combination of random under-sampling and augmentation-based over-sampling — is critical for achieving fair and clinically reliable model performance. Transfer learning with pretrained backbones (EfficientNet-B0, MobileNetV2, ResNet101V2) substantially reduces training time and data requirements. The adaptive learning rate scheduler stabilises training and prevents overfitting. Future work could explore SMOTE-based synthetic generation, class-weighted loss functions, or self-supervised pretraining on medical images for further improvements.

References

[1] Smedsrud, P.H. et al., "Kvasir-Capsule, a video capsule endoscopy dataset," *Scientific Data*, vol. 8, no. 1, pp. 1–14, 2021.

[2] Tan, M. & Le, Q., "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," *ICML*, 2019.

[3] He, K. et al., "Deep Residual Learning for Image Recognition," *CVPR*, 2016.

[4] Sandler, M. et al., "MobileNetV2: Inverted Residuals and Linear Bottlenecks," *CVPR*, 2018.

[5] Chawla, N.V. et al., "SMOTE: Synthetic Minority Over-sampling Technique," *JAIR*, vol. 16, pp. 321–357, 2002.

[6] PyTorch Documentation, https://pytorch.org/docs/stable/index.html

[7] TensorFlow / Keras Documentation, https://www.tensorflow.org/api_docs